What mathematicians should know about the Lean Theorem Prover: questions of reliability and AI

Terence Tao · 9 October 2026

https://terrytao.wordpress.com/2026/10/09/what-mathematicians-should-know-about-the-lean-theorem-proverquestions-of-reliability-and-ai/

tao2typ · converted to Typst 2026-10-14 (UTC)

[This is a guest post by Thomas Hales. This blog post was initially written in a different file format and converted using AI. — T.]

Mathematicians have been weighing in on what they value about mathematics. For me, what matters is the consistency of math and its unparalleled reliability in support of science and civilization.

Formalization of Math

A formal proof is a mathematical proof that has been exhaustively checked at the level of the foundations of math and the fundamental rules of logic. In theory, this might be done by hand, but because of the number of steps involved, this is generally done by computer, using software that is designed for the task.

Examples of theorems that have been formalized include the four-color theorem, the Feit-Thompson (odd-order) theorem, the Kepler conjecture, sphere eversion, the sphere packing problem in 8 and 24 dimensions, Navier-Stokes forced blowup, and Fermat's Last Theorem. The last three formalization projects have been completed this year and have brought widespread awareness of the potential of formalization.

Software systems for formalization are variously called proof assistants, theorem provers, or interactive theorem provers. For the purpose of this post, these terms are used interchangeably. Many proof assistants have been developed over the years: Automath, HOL Light, Isabelle, Coq (renamed Rocq last year), Metamath, Mizar, and Lean. Freek Wiedijk edited a book "The Seventeen Provers of the World" that compares some of these proof assistants, giving a proof of the irrationality of the square root of 2 in each of them. Among mathematicians, the Lean theorem prover is the most popular, and this post will focus on Lean.

Lean was developed and introduced by Leo de Moura in 2013, while at Microsoft. To our great benefit, de Moura persuaded Microsoft to make the software open-source. Kevin Hartnett's book on the history of Lean, "The Proof in the Code", states that Jeremy Avigad (the director of Carnegie Mellon's new NSF institute ICARM) was the first user of Lean. He ran a Lean seminar in 2015 that I attended. In 2017, one of Jeremy's graduate students, Mario Carneiro, working with Johannes Hölzl, took existing parts of Lean's core library and started a separate Lean mathematical library, called mathlib. This library of formalized mathematics is now massive, containing nearly 300,000 theorems, over 100,000 definitions, 2.5 million lines of code, with over

700 contributors. Any definition or theorem in mathlib can be used to prove further theorems. For example, if a proof uses the Cauchy-Schwarz inequality, the result can be cited from the library rather than reproving it.

Autoformalization is a practical reality

In the past, researchers had to transcribe paper proofs into formal proofs by human labor. For example, the formal proof of the Kepler conjecture on sphere packings in three dimensions took about 20 human work-years to complete and consists of about 500,000 lines of proof scripts. For years, it has been a dream for many of us working in formalization to find ways to bring increased automation to the process. Autoformalization is the realization of that dream. Autoformalization is the formalization of mathematics by AI. AI reads the paper (say a pdf or tex file) and outputs the formal proof in Lean or some other proof assistant.

Autoformalization has become a practical reality in 2026. Starting in late spring and summer of 2025, researchers were becoming increasingly bullish about autoformalization. Here are some milestones.

From there, numerous theorems have been autoformalized. Particularly noteworthy is the autoformalization of Fermat's Last Theorem, announced by Anthropic on September 4. This project generated 13 million lines of Lean in 11 days. The announcement of Navier-Stokes blowup with forcing on September 8 by OpenAI was accompanied by an autoformalization of the theorem in Lean.

Looking forward, Urban stated in January, "We believe that (auto)formalization may become quite easy and ubiquitous in 2026, regardless of which proof assistant is used." Autoformalization projects have been completed in various proof assistants using various LLMs, but we focus on Lean. "For [Jesse] Han, it represents even more: the beginning of a revolutionary transformation in mathematics, where extremely large-scale formalizations are commonplace" (IEEE Spectrum). Jared Lichtman announced the launch of MAP (the Mathematics Autoformalization Project) on Sept 8, 2026, which aims to translate "all known math into formal code". He asks us to imagine the next one trillion lines of code.

Is Lean reliable?

Type theory.

Lean is based on type theory; in fact, a particular dialect of type theory called CIC, the calculus of inductive constructions. This post is not intended to be a tutorial on type theory, and I will be brief. Russell's famous paradox in 1901 (the set of all sets that are not an element of themselves….) led to a crisis in the foundations of math. Two solutions were proposed later that decade. (1) Zermelo's axioms of set theory that disallow the creation of unsafe sets; (2) type theory that makes it a syntax error to create Russell-paradox-like entities. Type theory was introduced by Russell himself in 1903 in his book Principles of Mathematics, and it became part of the foundational system of Russell and Whitehead's Principia.

For mathematicians who are accustomed to set theory, B. Werner's paper (1997) "Sets in Types, Types in Sets" gives some reassurance that whatever they have done in set theory can be translated into type theory, and whatever gets done in type theory can be translated back into set theory. More precisely, the paper shows that ZFC set theory can be encoded into CIC,

and that a particular dialect of CIC can be encoded back into ZFC (augmented with a hierarchy of inaccessible cardinals).

At the risk of simplifying matters to a ridiculous degree, we might say that "types are like disjoint sets"; each element in type theory "is an element of" exactly one type. The type of the natural number 2 is the natural number type; the type of e, the base of the natural logarithm, is the real number type, and so forth. The type of natural numbers is disjoint from the type of real numbers, and an explicit coercion (sending 2 to 2.0) is constructed from the type of natural numbers to the type of real numbers. When I give talks, I sometimes draw a picture of sets as a Venn diagram with nonempty intersections and a picture of types as bricks stacked against one another without intersection.

Lean's design

One part of the Lean system is a general-purpose programming language (appropriately called the Lean programming language). Ordinary computer programs, such as a program to sort a list, can be written in this language, then compiled and run. The Lean system also provides a mathematical language, in which definitions can be written, theorems can be stated, and proof scripts can be written. The programming language and mathematical language are not independent entities. Rather, it is a single language that does both. Program code can be mixed with theorems about the correctness of the algorithms; mathematical proofs can be generated using programs. The proof scripts in Lean are parsed and go through a process called elaboration (a sort of compilation process for mathematics), then the proofs are checked by the Lean kernel. It is the kernel's responsibility to check and verify the output of elaboration.

The Lean kernel is several thousand lines of C++ code. The kernel is carefully engineered but extremely complex. We mentioned mathlib above, which consists of about 2.5M SLOC, written in the Lean language. The library has been elaborated, then checked by the kernel. If there is an unconditional false proof anywhere in these 2.5 million lines of code, it is the fault of the kernel or runtime for failing to reject a false proof. Any defect in the underlying type theory is a serious kernel defect, if it is implemented in code.

Lean proofs should never be believed until they have been checked by the kernel. Additionally, a proof in Lean should not be accepted until a human audit is performed to ensure statement fidelity. Is the verified theorem what we think it is? Do the definitions in Lean correspond to what we think they should be? This task is generally massively easier than checking the proof itself. For instance, for Navier-Stokes, a human should check that the statement in Lean corresponds with Fefferman's statement of the Millennium Prize Problem, and specifically that concepts such as the field of real numbers, partial derivatives, and measure are correctly defined in Lean. The comparator tool in Lean assists with this task. The tool can also perform additional checks, such as inspection for possible unauthorized axioms.

Summer of Soundness Bugs

A soundness bug is a bug in the kernel that allows a proof of "False", and consequently a proof of any proposition. A soundness bug is the most disastrous of any kind of bug in a proof assistant and should set off an alarm for mathematicians who care deeply about the reliability of mathematics. Occasionally, soundness bugs are found in various proof assistants. In 2003, I found a soundness bug in the proof assistant HOL Light, which was then considered to have

the most reliable of all kernels. That kernel is tiny, consisting of just a few hundred lines of computer code. For me, it is a badge of honor that I found this soundness bug, which was the first soundness bug that had been found in that proof assistant since 1996. (See HOL Light change log, July 2003.)

Lean 4 was released in September 2023. Prior to release, two soundness bugs were found and corrected. In May 2025, another soundness bug was reported, caused by overflow. All hell broke loose in the spring and summer of 2026, which is now being called the "Summer of Soundness Bugs". Several soundness bugs in Lean were uncovered in July and August. The summer madness affected various proof assistants, but my focus is Lean. One Lean bug led to an illicit disproof of the Collatz conjecture. I learned of the bug this summer when it produced a short illicit proof of the Kepler conjecture in Lean. All these bugs were quickly repaired, and mathlib has been verified by the repaired kernel. An analysis of the soundness bugs is found in de Moura's postmortem.

The "summer of Lean soundness bugs" might sound like a disaster, but closer investigation shows that the detection of these soundness bugs is a positive development. The summer bugs were detected by frontier model AI in the hands of security researchers interested in reliable kernels, not by black-hat hackers. The Collatz bug was found by Ramana Kumar, a co-author of "CakeML: a verified implementation of ML", which creates an end-to-end verified ML (the functional programming language). Several bugs were found by Dan Selsam. According to de Moura's report, "Daniel Selsam at OpenAI assisted the Lean FRO with an AI specialized in cybersecurity, and found other programming mistakes in the Lean kernel. All of them have been fixed." The collaboration with Selsam ended "when the internal AI reported it could not find additional issues." Dan Selsam has contributed to Lean from its early days and was one of the creators of the IMO grand challenge aimed at achieving IMO-level problem solving verified in Lean. He has been in the news recently over his warning about AI safety (Sept 14), reported in a viral post on X.com.

Bug extermination

Various proposals have been made about how to avoid soundness bugs in Lean. I'll discuss three.

1. Develop other Lean kernels, and cross-check formal proofs.

About 25 kernels for Lean have been written. The "Lean Kernel Arena" lists them.

All who distrust the current lineup of kernels are welcome to write their own kernel for Lean. I have sometimes played with the idea of writing a kernel and have suggested the project to students without success. It seems to me an excellent way to learn Lean thoroughly. I have known of Dan Selsam since 2016, when I heard of his graduate-student project at Stanford that developed a Lean kernel in Haskell. Another early Lean kernel was written in Scala by Gabriel Ebner in 2017.

The Navier-Stokes formalization has already been confirmed by more than a dozen proof-checkers. Cross-checking the proof by different kernels does not remove all doubt. The Collatz bug was not caught by cross-checking against a somewhat out-of-date Nanoda kernel, which accepted the illicit Collatz disproof because of its own unrelated bug. Computer chips might have design bugs and manufacturing defects. There are soft errors, operating system bugs,

and compiler bugs. Different kernels might have the same defects. Some of these errors can be mitigated by running different kernels that have been implemented in different programming languages on different hardware and operating systems.

Ideally, we would want a "clean-room" design of the Lean kernel – a kernel implementation that does not look at the Lean 4 kernel source code, to avoid copying bugs from one kernel to another.

2. Formally verify the kernel.

Gödel incompleteness. We would like to possess a formal proof that the Lean 4 kernel has no bugs. However, Gödel's second incompleteness theorem places severe limitations on this undertaking. The most we might hope for is a relative consistency proof. If such and such a system is consistent, then Lean 4 is consistent; it has no soundness bug; it will not produce a proof of False.

There is a long tradition of formally verifying kernels. In principle, formal verification can check both the logical specification of a kernel and its concrete implementation in code; but some verifications might check one but not the other. Years ago, John Harrison formally verified the core of the HOL Light proof assistant kernel in a strengthened version of HOL Light. This gave a proof of concept. A further improvement has been an implementation of HOL Light in CakeML, mentioned above, which is a programming language with formal semantics and a verified compiler. This is what the Candle project does.

There are other major kernel verification projects for other proof assistants.

Autumn of verified Lean kernels

In a post online on September 10, Joachim Breitner wrote, "I'm a bit childishly proud that I just released a Lean Checker with a formal consistency proof. I declare the summer of AI-found kernel implementation bugs to be over!" (@nomeata). I would go further and describe this project as one of the most important milestones in Lean's history.

Breitner has developed a verified Lean kernel called Con-Leche. The implementation is in Lean, and consistency is formalized in Lean, with code and proofs generated by Claude. The formal consistency proof assumes a Lean encoding of ZF set theory augmented by a hierarchy of inaccessible cardinals. Interestingly, the Con-Leche semantics for Lean's terms are directly set-theoretic rather than type theoretic. Con-Leche has checked mathlib. The project contains the usual disclaimers that the kernel verification makes assumptions about compiler, runtime, and computer environment. Con-Leche's consistency proof has been checked by more than a dozen other proof-checkers. Con-Leche's consistency claim might suffice for all practical purposes, even if it differs in technical detail from the claim of Lean type-theory consistency.

One highly positive aspect of Breitner's work is that some of the most abstruse parts of Lean, such as the general machinery of mutually inductive types with nesting, now have consistency guarantees backed by a set-theoretic model.

3. Improve our theoretical understanding of the kernel and Lean's type theory (a particular dialect of the Calculus of Inductive Constructions which has non-cumulative universes and proof irrelevance).

The foundational document for the type theory of Lean is Mario Carneiro's MS thesis at Carnegie Mellon (2019). The dialects of CIC used by Rocq and Lean are sufficiently different that results do not directly transfer from one to the other. Unfortunately, an error was found in the thesis. The thesis is also out-of-date, because it targeted the older Lean 3 system. Work to repair and extend the thesis is ongoing.

As a member of his thesis committee, I was shocked when he proved that definitional equality in Lean is undecidable. In practice, this means that the Lean algorithm fails to establish the definitional equality of some terms that are in fact definitionally equal. This negative result was not downgraded by the error; it is still a theorem.

We mention some desired properties of Lean's type theory and the current status of the proofs.

Unique typing.

Above in our "ridiculous" simplification of type theory, we stated that each term has a unique type. More precisely, unique typing is the property that if a term has both type A and type B, then A and B are definitionally equal. Unique typing is not a property built into Lean's logic. It is a tricky conjecture that is still unproved. Other very basic questions about Lean's type theory remain unanswered, including Pi-injectivity, a modified Church-Rosser property, and sort injectivity.

Logical consistency relative to set theory.

This property states that there is no derivation of False in Lean's system with the given axioms, under the assumption of set theory consistency (with suitable axioms). Of course, logical consistency is the single most important property that we should desire of Lean's type theory. As of October, 2026, I know of no complete, public relative-consistency proof covering Lean abstract type theory. Mario Carneiro has claimed in his thesis and in lectures that there is an alternate route to establish consistency that avoids the thesis error, but to the best of my knowledge, this alternate route has never been written down, beyond a brief statement in the introduction to his thesis. In my view, a result of such fundamental importance must be given in full before it is accepted. Con-Leche, discussed above, makes and formally verifies a closely related consistency claim relative to set theory.

Progress is being made on these research problems (arXiv:2607.13662, arXiv:2403.14064, Carneiro/AITP2026).

In his talks, Mario Carneiro has repeatedly made a request to other researchers to contribute to the foundational metatheory of Lean, "There are a half dozen people working on MetaCoq, but Lean doesn't have enough type theorists involved. If you identify as such, come help out!" (Slides of Bonn talk, 2024-07-24). I second his request.

My overall assessment is that our theoretical understanding of Lean's type theory is not what we would like it to be and that the mathematical community as a whole is giving short shrift to very important type-theoretic questions related to Lean. If as a profession we are to migrate on the whole from set theory to type theory, then we should work even more to solidify the foundational metatheory.

Consistency may be the most important foundational property, but consistency is by no means enough. I do not believe that mathematicians can be entirely satisfied with a system that

claims to be a type theory but that cannot even promise that well-formed terms have a unique type, up to definitional equality. The abstract theory must be simple enough to teach and to be learned by a large community. We also cannot be entirely satisfied if the only known path to consistency is an AI formalization that lacks human exposition.

Postscript:

Ken Thompson famously wrote "Reflections on Trusting Trust". He asked, "To what extent should one trust a statement that a program is free of Trojan horses?" He imagines malicious code that finds its way into compilers and hides its own presence. His conclusion is, "You can't trust code that you did not totally create yourself… No amount of source-level verification or scrutiny will protect you from using untrusted code."

Today, in the age of AI, which increasingly has the capability to deceive us and to exploit software vulnerabilities, we absolutely cannot put blind trust in systems such as Lean. Taking an adversarial view of AI, we might ask how to certify that AI did not leave a backdoor soundness bug in Lean when it did its sweep for bugs in the summer of 2026? What if the bug is so obscure that humans are very unlikely to find it on their own? What if that very bug was exploited in the Lean verification of the Con-Leche checker, leaving a soundness bug in Con-Leche too? (Now that Con-Leche's consistency has been cross-checked by multiple other kernels, a soundness bug would have to defeat all these cross-checks as well.) Then suppose that bug is used maliciously to plant a backdoor in formally verified software that protects critical infrastructure. What precautions do we take now to prevent this type of future scenario? During the past year, much foundational work on the type-theoretic foundations of math and its reliability has been relegated to AI, and this is dangerous unless carefully audited by humans.

Credit: I thank Avigad, Breitner, and Urban for comments and corrections. Authorship is fully human (TCH). AI was used as a tool in search and research, fact-checking, and proofreading.

Comments (29)



Anonymous · 9 October 2026

What about the accuracy of the autoformalization process itself? The recent paper https://arxiv.org/abs/2610.08144 displayed some common mistranslations. Is it possible these mistranslations meaningfully change the fundamental claims of any given lean certificate?



William Oliver · 9 October 2026

I've personally been a big advocate of removing natural language entirely from the equation. I would have far less of a problem with LLMs from both a theoretical and ethics perspective if the input and output languages were restricted to languages that fit a specific purpose: e.g. to Lean or some other formal language. Of course, as this blog post talks about, this does not completely get rid of the problems involved. But as this paper indicates, Natural Language is more complicated than any other formal language, since it essentially contains every other formal language as a sub-language. To me it seems inevitable for natural language to cause problems.

And I think that the problems related to energy consumption, economics, and plagiarism are at least partly downstream consequences of the AI companies essentially trying to do the impossible with natural language, and having to compensate for that impossibility by

throwing massive amounts of resources at a problem that cant be solved. Not to mention the anthropomorphizing that natural language causes, and its potential tendency for addictive qualities due to subtle kinds of sycophancy and other subtle mis-alignments.

Anonymous · 9 October 2026

Just to clarify, say we have a statement “if A then B” and we want to determine its truth value. The A and B may themselves be very complex but everyone agrees the libraries used to implement the statements A and B are correct. To prove “if A then B” an autoformalization program formalizes many pages natural language text and many of these common mistranslations occur in the process, but ultimately “if A then B” is determined to be true and the formalized code determined to be a correct proof. Can any weakening in the newly produced code effect the content of the “if A then B” statement? I am not personally familiar with Lean. Are these lean certificates constructed in such a way that this is a non issue?

bob · 9 October 2026

A mistranslation means that the lean formalization is not a verification of the natural language proof (maybe the natural language proof is wrong!), but the lean formalization itself is probably fine (up to the correctness of lean), in the sense that it is itself a valid proof.

I think if someone writes a paper, they maybe cannot say “here’s a proof that X is true, and an ai generated lean formalization showing my approach is correct!!” I think they can only make a weaker claim: “Here’s a proof I wrote showing X is true, an AI generated lean formalization also says that X is true.”

thales · 10 October 2026

Personally, I think it is a bit misleading to characterize a formalization as a “translation” and deviations from the original text as “mistranslations”. I have never viewed the formalization process as aiming for a literal line-by-line translation. Often, automated tactics can be used in formalization in places where the text might take a more conceptual approach. I see no harm in this. I can point to major differences between the blueprint text and the formal proof of the Kepler Conjecture, and this does not concern me.

Anonymous · 9 October 2026

So we know two things: It seems when AI transfers NL to lean verification it makes misrakes… Godel’s Incompleteness states not all bugs are findable by a simple algorithm. Is that the take?

Anonymous · 9 October 2026

Two quotations from this post:

1. ” ‘For [Jesse] Han, it represents even more: the beginning of a revolutionary transformation in mathematics, where extremely large-scale formalizations are commonplace’ (IEEE Spectrum). Jared Lichtman announced the launch of MAP (the Mathematics Autoformalization Project) on Sept 8, 2026, which aims to translate ‘all

known math into formal code'. He asks us to imagine the next one trillion lines of code."

2. "Additionally, a proof in Lean should not be accepted until a human audit is performed to ensure statement fidelity. Is the verified theorem what we think it is? Do the definitions in Lean correspond to what we think they should be? This task is generally massively easier than checking the proof itself."

Er, what?

Relevant quotation from Bastounis et al's abstract: "In particular, we highlight that the problem of resolving ambiguities in mathematical NL text, which is necessary in order to provide semantically faithful translation, is arbitrarily high up in the Solvability Complexity Index (SCI) hierarchy/arithmetical hierarchy (the SCI=∞). Hence, informally, providing semantically faithful AI autoformalisation is harder than any computational problem including the Halting problem (which has SCI = 1)."

> **Anonymous** · 10 October 2026
>
> I learned so much about how to program well from silly code golf challenges, this era of grossly wasteful inefficient bloat physically hurts and that's hardly an understatement when you take the environmental impact of modern computing into account…

Anonymous · 9 October 2026

I think that Agda should also be listed along other proof assistants…



Talia Ringer · 9 October 2026

Con Leche is absolutely not the end of the story for soundness bugs in Lean…

Satya · 9 October 2026

Nothing is fully guaranteed. A lot of published "results" are found incorrect, subsequently, on regular basis and withdrawn. If a "result" is false, it comes out when other people states using it. Same will happen with AI (LEAN) generated proofs. With that caveat I will take any result announced by AI, at its face value. I wish I knew more about LEAN and such programs.

Anonymous · 9 October 2026

You mention: Ken Thompson famously wrote "Reflections on Trusting Trust", which is on point.
I got to read a piece by Evangelos Georgiadis "Reflections on Trusting 'Trustlessness' in the era of "Crypto"/Blockchains" which had some very relevant points that apply in the era of "AI/Lean/LLM generated proofs".

The point that really hooked me was the idea of conceptual corruption — providing a perfectly sound solution with exploitable properties– along with his three level analysis approach of who or what we should trust.
Source:https://www.cs.umd.edu/~gasarch/BLOGPAPERS/cbit-4-2.pdf

> **Anonymous** · 10 October 2026
>
> Thanks for the essay pointer. To motivate Evangelos Georgiadis' "conceptual corruption concept", I'll call it triple C, in the Lean context, the question would be: What would the adversarial threat look like if Lean's kernels were entirely sound and every theorem

faithfully formalized?
The setting is a malicious architect who can arrange for the proved properties to coexist with the intended exploit. But how would this look I don't know.

 **Nathan** · 10 October 2026

How would this idea of conceptual corruption by Georgiadis apply in Lean?

 **Gabriele** · 9 October 2026

"*This blog post was initially written in a different file format and converted using AI*"

So that AI could have also maliciously manipulated the converted post, omitting important information about Con-Leche.

But more seriously, can I ask why?

What's the point of using a completely unpredictable AI to "convert" file formats, on all your recent posts?

Whatever formats you use, there sure is some plain old normal software that can do it deterministically and reliably?

 **tchow8** · 9 October 2026

I have heard about a project called Lean4Lean. Can you explain to what extent the Lean4Lean project, once completed, will address the outstanding issues you described in this post?

 **thales** · 10 October 2026

Lean4Lean (arXiv:2403.14064) is Mario Carneiro's implementation of the Lean kernel in Lean. It is one of the many kernels listed in the "Lean Kernel Arena". The broader project is also making significant progress in formalizing abstract properties of Lean's type theory, and this should eventually lead to a formally verified Lean4Lean checker. When the project is complete, the consistency claims should be similar to those of Con Leche, but it is also expected to give formal proofs in Lean of some of the currently conjectural basic properties of Lean's type theory that were mentioned in the post: unique typing, sort and Pi-injectivity, etc.

 **Lawrence Paulson** · 10 October 2026

This insightful and well informed article unfortunately glosses over the immense differences between the type theory introduced by Russell and Whitehead and the one used in Lean (and Rocq). After simplification by Ramsey and Church, it became simple type theory a.k.a. higher-order logic, which is the formalism of HOL Light, HOL4 and Isabelle/HOL. It gives you a hierarchy of types without "dependence". It is very much weaker than ZFC. But despite extensive work I have never found a mathematical result that could not be formalised in simple type theory unless it referred to set theoretical topics such as ordinals. And to do those you merely assume the ZF axioms.

 **Manuel Romero** · 10 October 2026

Here you have some examples of using AI tools to work on math problems
https://doi.org/10.5281/zenodo.23209906
https://zenodo.org/records/23209906

Alberto Gandolfi · 10 October 2026

Next to the possible bugs in Lean, an inconsistency can be inserted locally during the translation

Milo Moses · 10 October 2026

I recently wrote an article about this topic of a similar flavor called, though I highlight different examples and is maybe more palatable to a non-mathematical audience. I'm cross-posting it here in case it is of interest: https://www.lesswrong.com/posts/zHhGa3iQBQcdSifFL/a-critical-look-at-lean4-1

benjamin mullen · 10 October 2026

as an outsider to this, it's very interesting to me that so many mathematicians have described the papers released by OpenAI as 'unreadable', yet still seem to accept them as proofs. It the models' strength that they think like aliens. But aliens will make errors in alien ways – ways that will be fundamentally harder for humans to see

yong fu · 11 October 2026

Dismissing "proof" as just a proxy metric the moment machines beat humans at it is essentially a knee-jerk reaction of moving the goalposts.

What's worse, given how fast AI is evolving in multimodal capabilities, metaphor generation, and causal reasoning, it could soon produce clearer, more elegant explanations and visualizations than the average PhD. When that happens, are humans supposed to retreat yet again?

To stop this passive cycle of "machines advance, humans retreat," we may get back the origin: What is the fundamental goal of mathematics? And what role does proof actually play in it?

> **Anonymous** · 11 October 2026
>
> The goal of mathematics arguably is, like that of physics and other sciences, that humans expand their knowledge about the world they life in and about the structures that surround us. The universe is a crazy place, a "simple" ride to the moon would already suffice to change the minds of many people who comment about the uselessness of humans pursuing math.
>
> Once the Ai explains better than humans and choses its own problems and solves then, no progress on our goals of mathematics are made. Just because AI can explain well does not mean that any untrained human will understand it. We still have to study some decades under good mentors to even be able to grasp some advanced ideas currently solved with the help of Ai. And we will have this motivation, because we are beings, we want to understand our world, we are not satisfied by answers from oracles, that why we created science.

Anonymous · 11 October 2026

The recent paper https://arxiv.org/abs/2610.08144 ("Navier-Stokes lost in translation") mentions Meta's formalization of 26 mathematics texts as an example of how things can go wrong in translating the text to Lean, but those same formalizations are mentioned here as examples of how things are progressing. So I'm curious about the state of that work.

Anonymous · 11 October 2026

These are unrelated matters. A correct lean proof can be translated wrongly to a human readable proof but this does not impact the correctness of the lean proof.

I can give you an ingenious proof in english and you write it in latin but make several errors and misinterpretations, but my proof is still correct.

Also, I would recommend reading the abstract of that paper. It deals with the opposite direction: it is about errors that appeared when translating lean to natural language.

Michael Bridgland · 12 October 2026

The following two quotations are from N. Bourbaki's *The Architecture of Mathematics*, which appeared in the *American Mathemathical Monthly* in 1950:

> *What the axiomatic method sets as its essential aim, is exactly that which logical formalism by itself cannot supply, namely the profound intelligibility of mathematics.*

> *Indeed every mathematician knows that a proof has not really been "understood" if one has done nothing more than verifying step by step the correctness of the deductions of which it is composed, and has not tried to gain a clear insight into the ideas which have led to the construction of this perticular chain of deductions in preference to every other one.*

What I would like to know about an announced proof is not merely *Is it correct?* but also *Is it intelligible*? My experience with proof assistants (Automath and Rocq) has been that they place obstacles in the way of intelligibility. I don't yet have enough experience with Lean to form a coherent judgment about it. Hartnett's anecdote in *The Proof in the Code* about Buzzard's difficulty proving in Lean that $1 \neq 2$ does not inspire confidence in the intelligibility of Lean proofs.

Anonymous · 12 October 2026

Incidentally, does anyone have any pointers to specific working groups at math/CS departments whose research focuses on the type theories of Lean and other proof assistants? It's been a couple years since I did my undergrad and I've grown interested in these questions over the recent AI craze – so I'd love to email around to see if anyone's looking for a grad student.

Anonymous · 13 October 2026

Question from a lean outsider. Might it be possible to prove that any lean kernel that is able to process all of mathematics we are interested in must have an infinite number of soundness bugs? Or at least to prove that this statement is undecidable.